

Rapport de Projet

Microsoft Fabric

Pipeline de données de bout en bout

Ryma DRISS

M2 Data Engineer & Cloud

Table des matières

Introduction	2
1. Présentation du projet	2
1.1 Contexte	2
1.2 Données utilisées	2
2. Architecture mise en place	3
2.1 Le workspace	3
2.2 Les Lakehouses	3
3. Étape 1 : Ingestion des données dans Bronze	3
3.1 Travail réalisé	4
4. Étape 2 : Nettoyage des données dans Silver	5
4.1 Travail réalisé	5
5. Étape 3 : Modélisation dans Gold	7
5.2 Tables créées	7
6.Étape 4 : Modèle sémantique et mesures DAX	8
6.1 Création du modèle	8
7. Étape 5 : Création du rapport Power BI	8
8. Étape 6 : Pipeline d'orchestration	10
8.2 Chaînage	10
8.3 Test du pipeline	10
Conclusion	11
A Annexe : Structure du projet	12

Introduction

Dans ce projet, j'ai travaillé sur la réalisation d'un pipeline de données complet avec Microsoft Fabric. Le but était de partir de fichiers CSV bruts, de les nettoyer, de les transformer, puis de construire un modèle de données utilisable dans Power BI pour faire des analyses.

Ce projet m'a permis de voir toutes les étapes d'un traitement de données : l'ingestion, le nettoyage, la modélisation, la visualisation et l'orchestration. J'ai aussi appris à utiliser les principaux composants de Fabric comme les Lakehouses, les notebooks, les pipelines et le modèle sémantique.

1. Présentation du projet

1.1 Contexte

L'entreprise utilisée dans le sujet est *E-Shop Guinée*, une plateforme de e-commerce. Elle veut mettre en place un tableau de bord pour suivre les ventes, les produits, les clients et les magasins. Mon travail consistait à construire un pipeline de données de bout en bout dans Microsoft Fabric afin de transformer les données brutes en données prêtes pour l'analyse.

1.2 Données utilisées

Le projet repose sur 5 fichiers CSV :

Nom du fichier	Nombre de lignes	Description
commandes_brutes.csv	environ 17 500	contient les ventes brutes avec plusieurs anomalies
clients.csv	500	contient les informations des clients
produits.csv	50	contient les informations sur les produits
magasins.csv	8	contient les informations sur les magasins
calendrier.csv	731	contient les dates de 2024 et 2025

TABLE 1 – Fichiers utilisés dans le projet

Le projet couvre l'ensemble du cycle de vie de la donnée : ingestion, nettoyage, transformation, modélisation et restitution, le tout dans un seul Workspace Microsoft Fabric

2. Architecture mise en place

2.1 Le workspace

J'ai créé un workspace nommé :

WS_Projet_Ryma

C'est dans ce workspace que j'ai créé tous les éléments du projet.

2.2 Les Lakehouses

J'ai créé trois Lakehouses :

Lakehouse	Rôle
LH_Bronze	pour stocker les données brutes
LH_Silver	pour stocker les données nettoyées
LH_Gold	pour stocker les données finales pour l'analyse

	LH_Bronze	Lakehouse
	 LH_Bronze	Point de terminai...
	LH_Gold	Lakehouse
	 LH_Gold	Point de terminai...
	LH_Silver	Lakehouse
	 LH_Silver	Point de terminai...
	NB_01_Ingestion_Bronze	Bloc-notes
	NB_02_Bronze_to_Silver	Bloc-notes
	NB_03_Silver_to_Gold	Bloc-notes
	PL_ETL_Complet	Pipeline
	power_BI_1	Rapport
	power_bi_2	Rapport
	SM_Ventes	Modèle sémanti...

FIGURE 1 – Organisation générale du workspace

Étape 1 : Ingestion des données dans Bronze

1 Travail réalisé

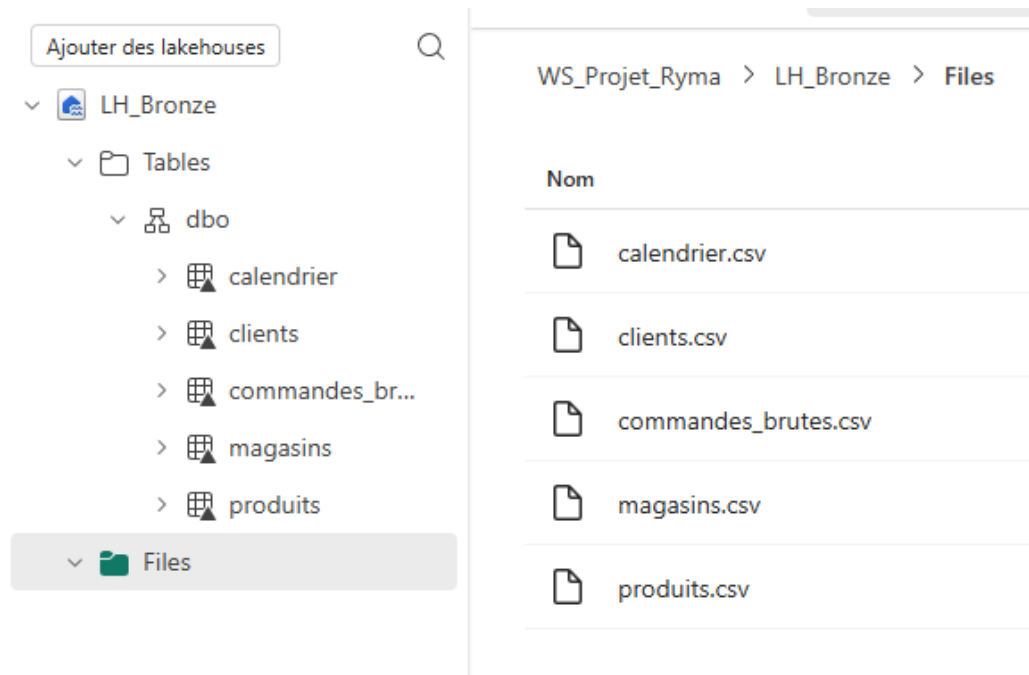
Dans cette étape, j'ai importé les 5 fichiers CSV dans le Lakehouse LH_Bronze. Ensuite, j'ai utilisé un notebook nommé NB_01_Ingestion_Bronze pour lire les fichiers CSV et les enregistrer sous forme de tables Delta.

- Le notebook lit les fichiers avec PySpark
- Puis les données sont enregistrées dans des tables dans le Lakehouse Bronze.

Listing 1 – code pour l'ingestion Bronze

```
df_cmd = spark.read.format("csv") \  
    .option("header", "true") \  
    .option("delimiter", ";") \  
    .option("encoding", "UTF-8") \  
    .load(  
        "abfss://WS_Projet_Ryma@onelake.dfs.fabric.microsoft.com/"  
        "LH_Bronze.Lakehouse/Tables/dbo/commandes_brutes"  
    )  
  
df_cmd.write.format("delta") \  
    .mode("overwrite") \  
    .save(  
        "abfss://WS_Projet_Ryma@onelake.dfs.fabric.microsoft.com/"  
        "LH_Bronze.Lakehouse/Tables/dbo/commandes_brutes"  
    )
```

À la fin de cette étape, les cinq tables ont été créées dans LH_Bronze.



4. Étape 2 : Nettoyage des données dans Silver

Travail réalisé

Dans cette étape, j'ai créé un notebook nommé NB_02_Bronze_to_Silver. Ce notebook sert à nettoyer la table `commandes_brutes` pour produire une table plus propre appelée `commandes_silver`. Les principales transformations réalisées sont :

- suppression des doublons
- suppression des lignes où `client_id` est vide
- suppression des lignes où `produit_id` est vide
- suppression des lignes où la quantité est inférieure ou égale à 0
- filtrage des dates invalides
- conversion de certaines colonnes en entier
- recalcul du montant net
- extraction de la date seule depuis la colonne `date_commande`

```
from pyspark.sql.functions import col, substring, trim
from pyspark.sql.types import IntegerType

df = spark.read.format("delta").load(
    "abfss://WS_Projet_Ryma@onelake.dfs.fabric.microsoft.com/"
```

```
    "LH_Bronze.Lakehouse/Tables/dbo/commandes_brutes"
)

df_clean = (
    df
    .dropDuplicates(["commande_id", "ligne"])
    .filter(
        (col("client_id").isNotNull()) &
        (trim(col("client_id")) != "")
    )
    .filter(
        (col("produit_id").isNotNull()) &
        (trim(col("produit_id")) != "")
    )
    .filter(col("quantite").cast("int") > 0)
    .filter(
        col("date_commande").rlike("^20(24|25)-\\d{2}-\\d{2}")
    )
    .withColumn("quantite", col("quantite").cast(IntegerType()))
    .withColumn("prix_unitaire", col("prix_unitaire").cast(IntegerType()))
    .withColumn("montant_brut", col("montant_brut").cast(IntegerType()))
    .withColumn("remise_pct", col("remise_pct").cast(IntegerType()))
    .withColumn(
        "montant_net",
        (
            col("montant_brut") *
            (1 - col("remise_pct") / 100)
        ).cast(IntegerType())
    )
    .withColumn(
        "frais_livraison",
        col("frais_livraison").cast(IntegerType())
    )
    .withColumn(
        "date_commande_date",
        substring("date_commande", 1, 10)
    )
)

df_clean.write.format("delta").mode("overwrite").save(
    "abfss://WS_Projet_Ryma@onelake.dfs.fabric.microsoft.com/"
    "LH_Silver.Lakehouse/Tables/dbo/commandes_silver"
)
```

Après ce traitement, la table `commandes_silver` contient des données plus propres et plus cohérentes. elle a ensuite été utilisée pour construire le niveau Gold.

5. Étape 3 : Modélisation dans Gold

Travail réalisé

Dans cette étape, j'ai créé le notebook `NB_03_Silver_to_Gold`. Son rôle est de construire le schéma en étoile à partir de la table `commandes_silver` et des dimensions présentes dans le niveau Bronze.

5.2 Tables créées

Les tables créées dans `LH_Gold` sont :

- `fait_ventes`
- `dim_client`
- `dim_produit`
- `dim_magasin`
- `dim_date`

Principe

La table de faits contient les informations de vente, tandis que les dimensions contiennent les informations descriptives.

```
fait = df_silver.select(  
    "commande_id", "ligne",  
    col("date_commande_date").alias("date_id"),  
    "client_id", "produit_id", "magasin_id",  
    "quantite", "prix_unitaire", "remise_pct",  
    "montant_brut", "montant_net", "frais_livraison",  
    "moyen_paiement", "statut_commande", "canal_vente"  
)
```

Cette étape permet d'obtenir un modèle propre et bien structuré pour l'analyse dans Power BI.

Étape 4 : Modèle sémantique et mesures DAX

Création du modèle

À partir du SQL Endpoint de LH_Gold, j'ai créé un modèle sémantique en sélectionnant les tables du schéma en étoile.

Relations créées

- fait_ventes.client_id vers dim_client.client_id
- fait_ventes.produit_id vers dim_produit.produit_id
- fait_ventes.magasin_id vers dim_magasin.magasin_id
- fait_ventes.date_id vers dim_date.date_id

Mesures DAX

J'ai créé les mesures suivantes :

CA Total = SUM(fait_ventes[montant_net])

Nb Commandes = DISTINCTCOUNT(fait_ventes[commande_id])

Panier Moyen = DIVIDE([CA Total], [Nb Commandes], 0)

Quantite Totale = SUM(fait_ventes[quantite])

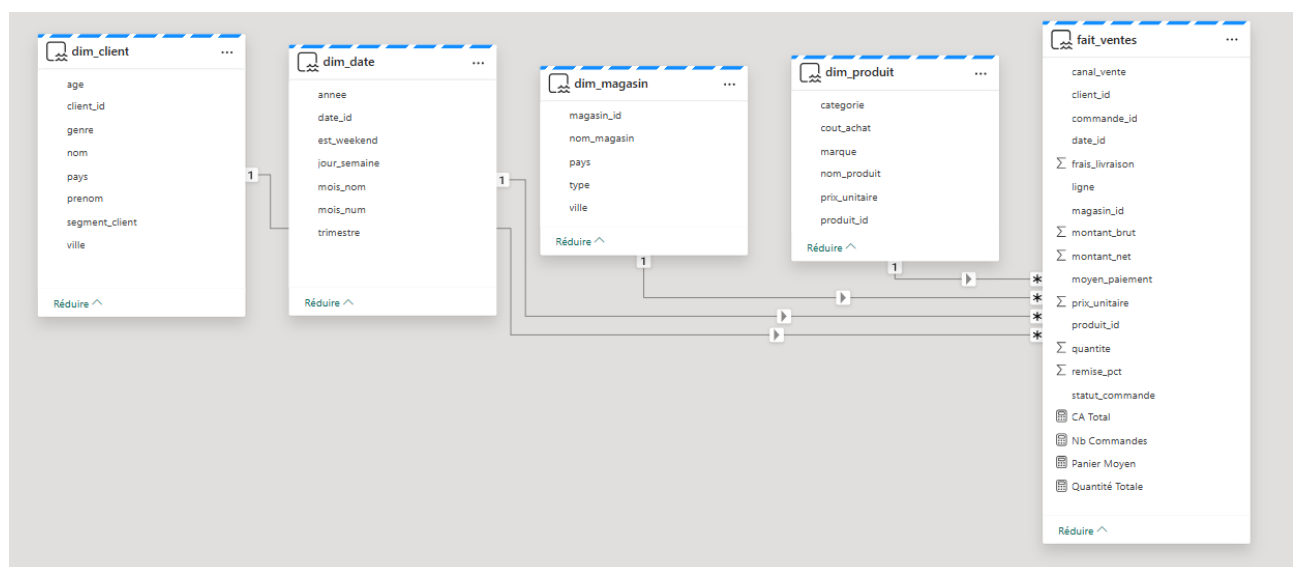


FIGURE 2 – modèle sémantique avec les relations

Étape 5 : Création du rapport Power BI

Dans la première page du rapport, j'ai ajouté :

- le chiffre d'affaires total
- le nombre de commandes
- le panier moyen
- la quantité totale
- un graphique du chiffre d'affaires par catégorie
- une courbe du chiffre d'affaires mensuel
- un filtre par année

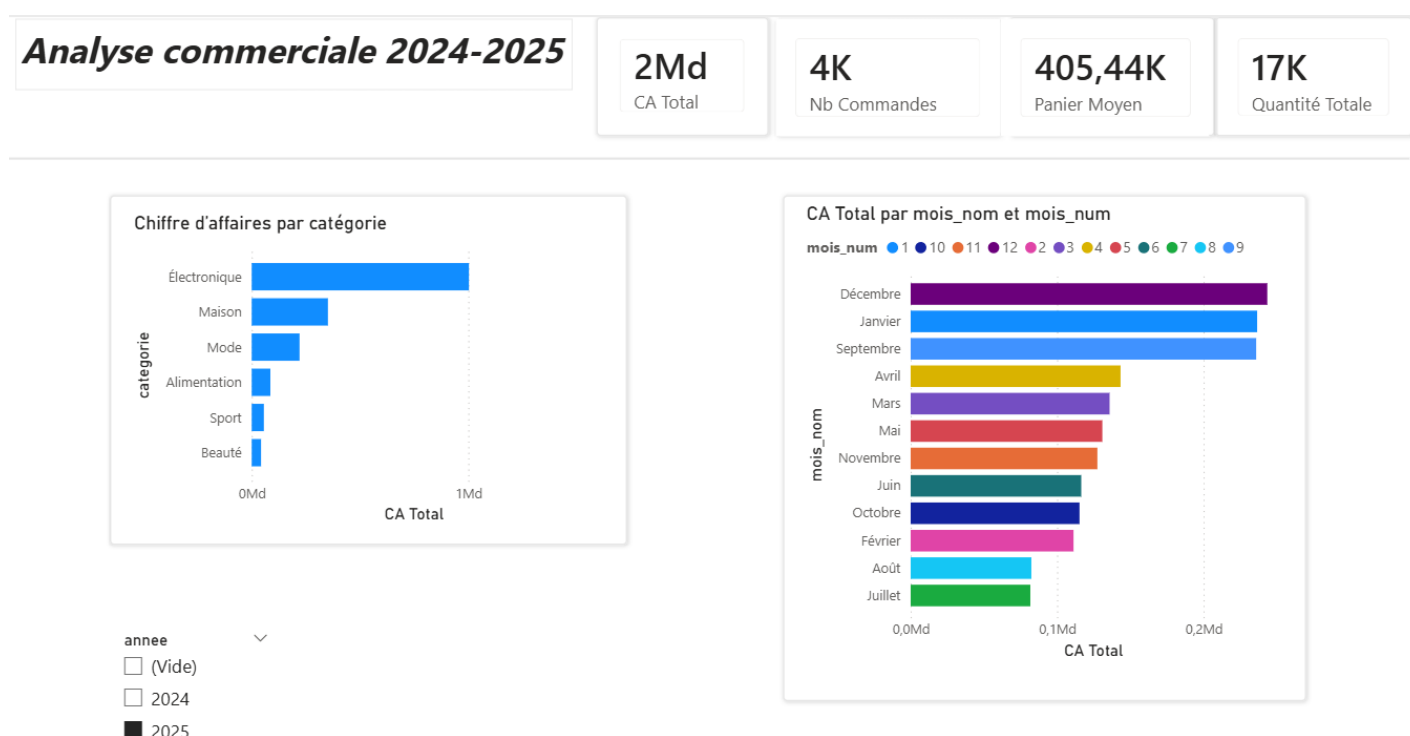


FIGURE 3 – Power BI - page 1

Dans la deuxième page, j'ai ajouté :

- le top 10 des produits par chiffre d'affaires
- la répartition du chiffre d'affaires par pays
- le chiffre d'affaires par magasin
- un filtre par catégorie

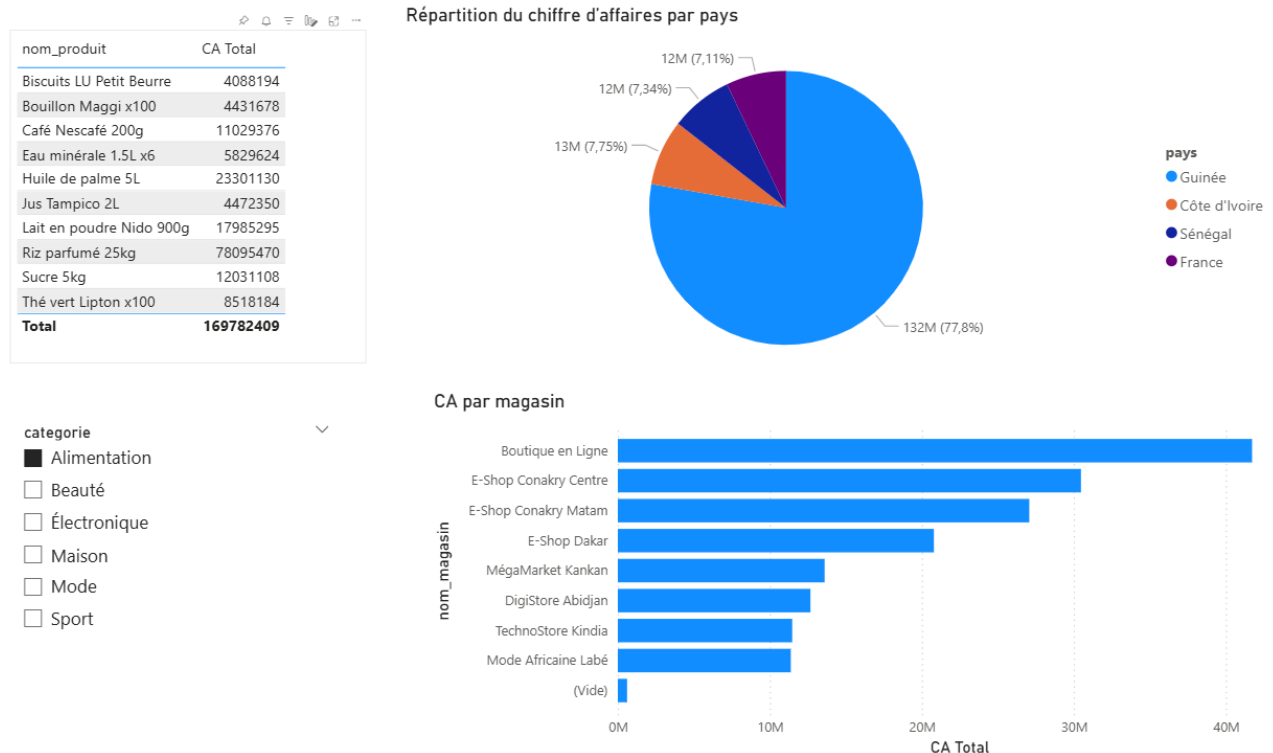


FIGURE 4 – Power BI - page 2

8. Étape 6 : Pipeline d'orchestration

Travail réalisé

J'ai créé un pipeline nommé **PL_ETL_Complet** dans le workspace. Ce pipeline contient trois activités notebook :

- NB_01_Ingestion_Bronze
- NB_02_Bronze_to_Silver
- NB_03_Silver_to_Gold

Chaînage Les notebooks ont été reliés avec la dépendance **On Success**. Cela veut dire que :

- NB_02 démarre seulement si NB_01 réussit
- NB_03 démarre seulement si NB_02 réussit

Test du pipeline Le pipeline a été testé avec l'option **Debug/Run**. Pendant les essais, j'ai rencontré une erreur liée aux limites de capacité Spark avec le message **TooManyRequestsForCapacity**.

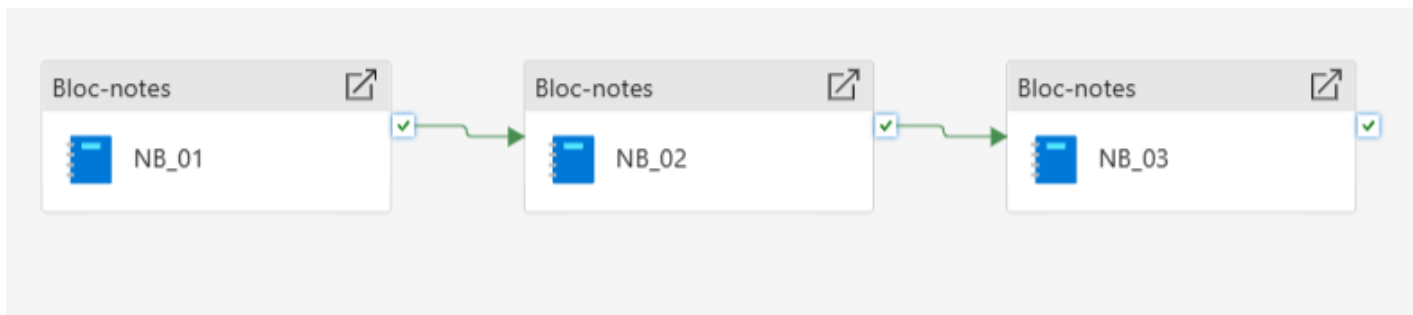


FIGURE 5 – Pipeline

Remarque : le pipeline ne s'exécute pas entièrement à cause d'une erreur de capacité Spark, et non à cause d'une erreur dans le code. J'ai testé les notebooks individuellement, et ils fonctionnent correctement. Le problème apparaît surtout lors de l'exécution complète du pipeline, ce qui est lié à une limitation temporaire des ressources Spark disponibles.

Réglages

Variables

Paramètres

Sortie

Variables de bibliothèque

ID d'exécution de pipeline

58427c78-06f0-4c52-a612-40b306a0b580

État du pipeline

✖

Échoué

⬇

 Exporter au format CSV | ⌵

☰

 Filtrer ⌵

✎

 ⌵

🔍 Filtrer par mot clé

Affichage des éléments 1-3

Nom de l'activité ⬆⬇	Statut de l'activité ⬆⬇	Début de l'exécution ⬆⬇	Durée ⬆⬇	Entrée	Sortie
NB_03	✖ Échoué	3/18/2026, 2:15:34 PM	52s	📄	📄
NB_02	✔ Opération réussie	3/18/2026, 2:14:25 PM	1m 8s	📄	📄
NB_01	✔ Opération réussie	3/18/2026, 2:13:23 PM	1m 1s	📄	📄

FIGURE 6 – Pipeline

Conclusion

Ce mini-projet m'a permis de mieux comprendre le fonctionnement de Microsoft Fabric et de manipuler plusieurs outils importants de la chaîne de traitement des données. J'ai appris à charger des fichiers CSV, à nettoyer les données avec PySpark, à créer un schéma en étoile, à construire un modèle sémantique et à réaliser un rapport Power BI.

J'ai aussi compris l'importance de l'orchestration avec un pipeline, ainsi que les difficultés possibles liées aux dépendances, aux contextes de Lakehouse et à la capacité Spark.

En résumé, ce projet a été une bonne mise en pratique de la logique de Data Engineering sur le cloud.

A. Annexe : Structure du projet

WS_Projet_Ryma

LH_Bronze

LH_Silver

LH_Gold

NB_01_Ingestion_Bronze

NB_02_Bronze_to_Silver

NB_03_Silver_to_Gold

PL_ETL_Complet

Semantic Model

Rapport Power BI